

Университет ИТМО

Факультет Программной Инженерии и Компьютерной техники

Информатика
Лабораторная работа №2
Синтез помехоустойчивого кода
Вариант №9

Выполнил:
Джейранов Георгий Ксенофонович

Группа Р3119

Проверил:
Осипов В.В

Санкт-Петербург
2026

Содержание

1.....	Содержание
2.....	Задания
3-6.....	Выполнение заданий
7.....	Заключение
8.....	Список используемых источников

2. На основании номера варианта задания выбрать набор из 4 полученных сообщений в виде последовательности 7-символьного кода.
3. Построить схему декодирования классического кода Хэмминга (7;4), которую представить в отчёте в виде изображения.
4. Показать, исходя из выбранных вариантов сообщений (по 4 у каждого – часть №1 в варианте), имеются ли в принятом сообщении ошибки, и если имеются, то какие. **Подробно прокомментировать** и записать правильное сообщение.
5. На основании номера варианта задания выбрать 1 полученное сообщение в виде последовательности 15-символьного кода.
6. Построить схему декодирования классического кода Хэмминга (15;11), которую представить в отчёте в виде изображения.
7. Показать, исходя из выбранного варианта сообщений (по 1 у каждого – часть №2 в варианте), имеются ли в принятом сообщении ошибки, и если имеются, то какие. **Подробно прокомментировать** и записать правильное сообщение.
8. Сложить номера всех 5 вариантов заданий. **Умножить полученное число на 4.** Принять данное число как число информационных разрядов в передаваемом сообщении. Вычислить для данного числа минимальное число проверочных разрядов и коэффициент избыточности.
9. Дополнительное задание №1 (позволяет набрать от 86 до 100 процентов от максимального числа баллов БаРС за данную лабораторную). Сделать себе учётную запись на <https://gitlab.se.ifmo.ru/>.
10. Написать программу на любом языке программирования, которая на вход получает набор из 7 цифр «0» и «1», записанных подряд, анализирует это сообщение на основе классического кода Хэмминга (7,4), а затем выдает правильное сообщение (**только информационные биты**) и указывает бит с ошибкой при его наличии.

09	22	44	66	88	10
----	----	----	----	----	----

1.	22	1	0	0	0	0	0	1
----	----	---	---	---	---	---	---	---

1.	44	0	0	0	1	0	1	1
----	----	---	---	---	---	---	---	---

1.	66	1	0	1	1	1	0	0
----	----	---	---	---	---	---	---	---

1.	88	0	1	0	0	1	1	0
----	----	---	---	---	---	---	---	---

2.	10	0	1	1	1	0	0	0	1	0	1	0	0	0	0	0
----	----	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

№3

4 сообщения:

22-1000001

r_1	r_2	i_1	r_3	i_2	i_3	i_4	Сумма	Синдром	№ ошибочной позиции	Результат
1	0	0	0	0	0	1				
							2(чет)	0	$110_2 = 6_{10}$	100011
							1(нечет)	1		
							1(нечет)	1		

44-0001011

r_1	r_2	i_1	r_3	i_2	i_3	i_4	Сумма	Синдром	№ ошибочной позиции	Результат
0	0	0	1	0	1	1				
							1(нечет)	1	$101_2 = 5_{10}$	000111
							2(чет)	0		
							3(нечет)	1		

66-1011100

r_1	r_2	i_1	r_3	i_2	i_3	i_4	Сумма	Синдром	№ ошибочной позиции	Результат
1	0	1	1	1	0	0				
							1(нечет)	1	$011_2 = 3_{10}$	1001100
							1(нечет)	1		
							0(чет)	0		

88-0100110

r_1	r_2	i_1	r_3	i_2	i_3	i_4	Сумма	Синдром	№ ошибочной позиции	Результат
0	1	0	0	1	1	0				
							1(нечет)	1	$001_2 = 1_{10}$	1100110
							2(чет)	0		
							2(чет)	0		

№4

У нас есть проверочные биты(r -степени 2) и информационные биты(i -остальные) Сначала мы ищем синдром, чтобы понять № ошибочной позиции, для этого мы складываем сначала цифры через одну(одну берем одну пропускаем), начиная с первой позиции, потом мы складываем цифры через две(две берем две пропускаем) начиная со 2 позиции, потом мы складываем цифры через четыре(четыре берем четыре пропускаем) начиная с 4 позиции и так далее. Сколько чисел мы пропускаем и с какой позиции начинаем можно определить по степени 2, сначала мы брали числа через одну($2^0=1$), потом через два($2^1=2$), потом через четыре($2^2=4$). Далее мы берем все полученные числа в обратном порядке и переводим полученное число в десятичную систему счисления. Так мы вычислили № ошибочной позиции, и заменили эту позицию($0 \Rightarrow 1; 1 \Rightarrow 0$).

№5 ВАРИАНТ №2

10-011100010100000

r_1	r_2	i_1	r_3	i_2	i_3	i_4	r_4	i_5	i_6	i_7	i_8	i_9	i_{10}	i_{11}	Сумма	Синд- ром	№ ошиб. позиции	Результат
0	1	1	1	0	0	0	1	0	1	0	0	0	0	0				
															1(нечет)	1	$1110_2 = 7_{10}$	0110011 0100000
															3(нечет)	1		
															1(нечет)	1		
															2(чет)	0		

№6

У нас есть проверочные биты(r -степени 2) и информационные биты(i -остальные) Сначала мы ищем синдром, чтобы понять № ошибочной позиции, для этого мы складываем сначала цифры через одну(одну берем одну пропускаем), начиная с первой позиции, потом мы складываем цифры через две(две берем две пропускаем) начиная со 2 позиции, потом мы складываем цифры через четыре(четыре берем четыре пропускаем) начиная с 4 позиции, потом мы берем цифры через восемь(восемь берем восемь пропускаем) начиная с 8 позиции. Сколько чисел мы пропускаем и с какой позиции начинаем можно определить по степени 2, сначала мы брали числа через одну($2^0=1$), потом через два($2^1=2$), потом через четыре($2^2=4$), потом через 8($2^3=8$). Далее мы берем все полученные числа в обратном порядке и переводим полученное число в десятичную систему счисления. Так мы вычислили № ошибочной позиции, и заменили эту позицию($0 \Rightarrow 1; 1 \Rightarrow 0$).

№8

Число информационных разрядов:

$$(22+44+66+88+10)*4=920$$

Минимальное число проверочных разрядов:

$$2^r \geq k+r+1$$

$$k=920$$

Проверим для $r=9 \Rightarrow 2^9 \geq 920+10+1 \Rightarrow 512 \geq 931$ -не верно

Проверим для $r=10 \Rightarrow 2^{10} \geq 920+10+1 \Rightarrow 1024 \geq 931$ -все верно

Ответ:10

Общее число разрядов в сообщении:

$$n = k+r=920+10=930$$

Коэффициент избыточности:

$$X=r/n=10/930 \approx 0.01075(1.08\%)$$

Ответ:1.08%

№10

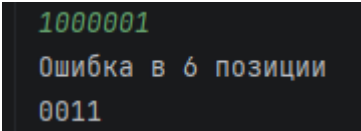
Исходный код: <https://github.com/Georg-Dzh/LabInf/blob/main/Lab2/Main.java>
<https://gitlab.se.ifmo.ru/Georgi-dzh/labinf/-/blob/master/Main.java>

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String n = scanner.nextLine();
        String[] chisla = n.split("");
        int[] bits = new int[7];
        for (int i = 0; i < 7; i=i+1) {
            bits[i] = Integer.parseInt(chisla[i]);
        }
        int s1 = (bits[0] + bits[2] + bits[4] + bits[6]) % 2;
        int s2 = (bits[1] + bits[2] + bits[5] + bits[6]) % 2;
        int s3 = (bits[3] + bits[4] + bits[5] + bits[6]) % 2;

        int sindrom = (s3*4 + s2*2 + s1*1) - 1;

        if (sindrom >= 0) {
            System.out.println("Ошибка в " + (sindrom + 1) + " позиции");
            if (bits[sindrom] == 0) {
                bits[sindrom] = 1;
            } else {
                bits[sindrom] = 0;
            }
        } else {
            System.out.println("Ошибок не обнаружено.");
        }
        System.out.println("" + bits[2] + bits[4] + bits[5] + bits[6]);
    }
}
```

Пример результата:



```
1000001
Ошибка в 6 позиции
0011
```

Заключение

В процессе лабораторной работы я познакомился с кодом Хэмминга, научился находить синдромы и строить схемы с его помощью. Понял как устроены информационные и проверочные биты.

Список используемых источников

Код Хэмминга

https://ru.wikipedia.org/wiki/%D0%9A%D0%BE%D0%B4_%D0%A5%D1%8D%D0%BC%D0%BC%D0%B8%D0%BD%D0%B3%D0%B0

Коды Хэмминга

<https://postnauka.org/video/44742>

Лекция 218. Код Хемминга

https://www.youtube.com/watch?v=wQLo4O2hNjI&xstg=CAMSEBUJ_b-oH-PhF0yjBgaukzY%3D

Код Хэмминга. Пример работы алгоритма

<https://habr.com/ru/articles/140611/>